Конспект лекции 7:
Языковые модели (LLM)

Липкин С.М.

1 Постановка задачи языкового моделирования

Языковое моделирование — фундаментальная задача NLP. Модель учится предсказывать распределение вероятностей последовательностей токенов.

1.1 Формальное определение

Для последовательности токенов $w_1, w_2, \ldots, w_n$ совместная вероятность раскладывается через цепное правило:

$$P(w_1, w_2, \ldots, w_n) = \prod_{t=1}^{n} P(w_t \mid w_1, \ldots, w_{t-1}) = \prod_{t=1}^{n} P(w_t \mid w_{<t}).$$

Задача языковой модели — научиться аппроксимировать $P(w_t \mid w_{<t})$ для любого контекста.

1.2 История

- **N-граммы:** считаем частоты n последовательных токенов. Не работают на длинных контекстах.
- **RNN/LSTM:** обрабатывают последовательность шаг за шагом, но медленно и с затухающими градиентами.
- **Transformer (2017):** параллельная обработка всей последовательности через self-attention.
- **LLM (2018–настоящее время):** Transformer-архитектуры на масштабе миллиардов параметров.

2 Encoder vs Decoder: два подхода

Все современные LLM используют архитектуру Transformer, но по-разному: encoder-only для понимания, decoder-only для генерации, encoder-decoder для трансформаций.

2.1 Encoder-only (BERT)

- Каждый токен видит полный контекст (бидирекционально).
- Нет каузальной маски — attention на все токены.
- Используется: классификация, NER, QA, NLI.
- Примеры: BERT, RoBERTa, ALBERT, DistilBERT.

2.2 Decoder-only (GPT)

- Каждый токен видит только предыдущие (causal mask).
- Авторегрессивная генерация: предсказываем следующий токен.
- Используется: генерация текста, диалог, код.
- Примеры: GPT, LLaMA, Mistral, Qwen.

2.3 Encoder-Decoder (T5, BART)

- Encoder читает вход (бидирекционально).
- Decoder генерирует выход (авторегрессивно, с cross-attention к encoder).
- Используется: перевод, суммаризация, преобразование текста.
- Примеры: T5, BART, PEGASUS.

3 Encoder-based модели: BERT

BERT (Bidirectional Encoder Representations from Transformers) предложен Google в 2018 году (Devlin et al.).

3.1 Архитектура

BERT использует многослойный Transformer encoder. Стандартные конфигурации:

- BERT-Base: 12 слоёв, 768 hidden, 12 heads = 110M параметров.
- BERT-Large: 24 слоя, 1024 hidden, 16 heads = 340M параметров.

3.2 Предобучение

BERT обучается на двух задачах одновременно:

1. Masked Language Modeling (MLM):

- 15% токенов выбираются для маскировки.
- Из выбранных: 80% заменяются на [MASK], 10% — на случайный токен, 10% — остаются без изменений.
- Модель учится восстанавливать исходные токены.

- Это позволяет учитывать контекст слева и справа (бидирекциональность).

2. Next Sentence Prediction (NSP):

- На вход подаются два предложения: A и B.
- Модель должна предсказать, является ли B следующим за A.

CLS -токен аккумулирует информацию для классификации.

- Позже выяснилось, что NSP мало эффективен (RoBERTa его удалила).

3.3 Fine-tuning

После предобучения BERT донастраивается под конкретную задачу:

- **Классификация:** [CLS] → линейный слой → softmax.
- **Token-level (NER, POS):** каждый выходной токен → линейный слой.
- **QA (SQuAD):** предсказание start/end позиции ответа.

3.4 Варианты и развитие

- **RoBERTa:** больше данных, нет NSP, динамическая маскировка.
- **ALBERT:** разделение параметров (factorized embedding + cross-layer sharing).
- **DistilBERT:** дистилляция (40% меньше, 97% качества).
- **ELECTRA:** детекция заменённых токенов вместо MLM.

4 Авторегрессионные модели: GPT

GPT (Generative Pre-trained Transformer) предложен OpenAI в 2018 году.

4.1 Архитектура и обучение

GPT использует decoder-only Transformer с каузальным маскированием. Функция потерь — стандартная кросс-энтропия для next-token prediction:

$$\mathcal{L} = -\frac{1}{T}\sum_{t=1}^{T} \log P(x_t \mid x_{<t}; \theta).$$

Обучение ведётся на огромных корпусах текстов (Common Crawl, WebText, Wikipedia, книги). Токенизация — BPE (Byte Pair Encoding, Sennrich et al., 2016), которая разбивает слова на подслова, решая проблему OOV.

4.2 Эволюция GPT

- **GPT-1 (2018):** 117M параметров, 12 слоёв. Предобучение + fine-tuning.
- **GPT-2 (2019):** 1.5B параметров, 48 слоёв. Zero-shot способности на многих задачах.
- **GPT-3 (2020):** 175B параметров, 96 слоёв. In-context learning (few-shot без fine-tuning).
- **GPT-4 (2023):** ≈1.7T параметров (предположительно MoE). Мультимодальность.

4.3 Инференс

Авторегрессивный инференс — последовательная генерация:

1. Подаём prompt.
2. Получаем распределение $P(x_{t+1} \mid x_{1:t})$.
3. Выбираем следующий токен (greedy, top-k, top-p, temperature).
4. Добавляем к последовательности, повторяем.

KV Cache: ключевая оптимизация. На каждом шаге пересчёт K и V для предыдущих токенов — дорого. KV-кеш сохраняет K и V из предыдущих шагов, на новом шаге считаем только для нового токена. Снижает сложность с $O(n^2)$ до $O(n)$ на шаг.

Стратегии декодирования:

- **Greedy:** $x_{t+1} = \arg\max P(\cdot \mid x_{1:t})$. Просто, но может зацикливаться.
- **Top-k:** ресемплинг из k наиболее вероятных токенов.
- **Nucleus (top-p):** выбираем минимальное множество токенов с кумулятивной вероятностью $\geq p$.
- **Temperature:** $P_i \propto \exp(\text{logit}_i/T)$. $T \to 0$ — argmax, $T \to \infty$ — равномерное.

4.4 Emergent abilities и Scaling Laws

Emergent abilities: способности, которые не проявляются в маленьких моделях, но внезапно появляются при масштабировании. Примеры: in-context learning, chain-of-thought, арифметика, перевод.

Scaling Laws (Kaplan et al., 2020): loss модели следует степенному закону от числа параметров N и объёма данных D:

$$\text{Loss}(N, D) \propto N^{-\alpha} \cdot D^{-\beta},$$

где $\alpha \approx 0.076$, $\beta \approx 0.103$. Вывод: увеличивать N выгоднее, чем D.

Chinchilla (Hoffmann et al., 2022): DeepMind показала, что предыдущие результаты недооценивали важность данных. Оптимальное соотношение:

$$N_{\text{opt}} \approx 2.1 \times 10^9 \cdot C^{0.5},$$

где C — compute budget в FLOPs. Для 175B модели нужно 3.7T токенов (у GPT-3 было 300B — сильный under-training). Chinchilla (70B на 1.4T токенов) превзошла GPT-3 при меньшем размере.

5 От GPT-3 к ChatGPT

5.1 Проблема raw GPT-3

- Умеет продолжать текст, но не следует инструкциям.
- Пример: `Переведи на русский: Hello` → `«Hello» может быть переведено...`
- Нужно: `Здравствуйте`

5.2 Instruction Tuning (SFT)

Собираем пары (инструкция, идеальный ответ) и дообучаем модель:

- **FLAN (Wei et al., 2021):** 60+ NLP-задач, переформулированных как инструкции.
- **Alpaca (Stanford, 2023):** 52K инструкций от GPT-3.5 через Self-Instruct.
- **OpenAssistant:** краудсорсинговые датасеты диалогов.
- **Формат:** system / user / assistant роли в промпте.

5.3 RLHF (Reinforcement Learning from Human Feedback)

Трёхшаговый процесс от InstructGPT (Ouyang et al., 2022):

Шаг 1: SFT — дообучение на инструкциях (как выше).

Шаг 2: Reward Model (RM) — обучаем модель ранжировать ответы:

- Генерируем несколько ответов на промпт.
- Человек ранжирует их (A > B > C).
- RM учится предсказывать предпочтения человека.
- Loss: Bradley-Terry model $P(A > B) = \sigma(r_A - r_B)$.

Шаг 3: PPO (Proximal Policy Optimization) — оптимизируем LM:

- Генерируем ответ → RM даёт награду → PPO обновляет LM.
- KL penalty: $\beta \cdot D_{KL}(\pi_{\text{PPO}} \parallel \pi_{\text{SFT}})$ — чтобы не отойти далеко от SFT-модели.
- Итоговая loss: $\mathcal{L} = \mathbb{E}[-r(x, y)] + \beta \cdot D_{KL}(\pi_{\text{PPO}} \parallel \pi_{\text{SFT}})$.

Ключевой результат InstructGPT: 1.3B модель с RLHF превзошла 175B GPT-3 по оценкам людей. Alignment важнее размера.

5.4 ChatGPT (ноябрь 2022)

ChatGPT добавил к RLHF:

- Диалоговый формат с multi-turn историей.
- Safety fine-tuning: отказ от вредных запросов.
- Контроль стиля и ограничений.
- Продвинутые методы инференса (latency/throughput оптимизации).

6 Альтернативные подходы к alignment

- **DPO (Direct Preference Optimization, 2023):** явно задаём предпочтения без RM и RL. Оптимизируем напрямую: $\mathcal{L} = -\mathbb{E}[\log \sigma(\beta(\log \pi_\theta(y_w|x) - \log \pi_\theta(y_l|x)) - \beta \log(\pi_{\text{ref}}(y_w|x)/\pi_{\text{ref}}(y_l|x)))]$.
- **GRPO (Group Relative PPO):** DeepSeek. Без Critic-сети. Награда нормируется по группе ответов.
- **Constitutional AI:** модель сама генерирует и оценивает свои ответы по конституции (набору правил).
- **RLAIF:** Reward Model заменяется на LLM, дающую AI feedback.

7 DeepSeek и Reasoning

7.1 DeepSeek-R1 (январь 2025)

DeepSeek-R1 показала, что рассуждение (reasoning) можно обучить через RL без SFT:

1. **Cold-start:** небольшой SFT на цепочках рассуждений.
2. **GRPO (Group Relative Policy Optimization):** награда за правильный ответ + формат reasoning.
3. **Self-evolution:** модель естественно учится более длинным и сложным рассуждениям.
4. **Aha-moment:** модель «перепроверяет» свои рассуждения — поведение, которого не было в обучении.

7.2 DeepSeek-V3 (декабрь 2024)

- 671B параметров, 37B активных (MoE с 256 экспертами).
- Обучен на 14.8T токенов. FP8 mixed-precision.
- Стоимость обучения: \$5.5M (в 10 раз дешевле аналогов).
- Использует Multi-Token Prediction (MTP) — предсказание нескольких токенов сразу.
- Multi-head Latent Attention (MLA) — сжатие KV кеша для эффективного инференса.

8 Mixture of Experts (MoE)

8.1 Идея

MoE заменяет плотные FFN-слои на спарсные наборы экспертов. Для каждого токена активируется только подмножество экспертов (обычно top-2):

$$\text{MoE}(x) = \sum_{i=1}^{E} G(x)_i \cdot E_i(x), \quad G(x) = \text{softmax}(W_g x),$$

где $G(x)$ — gating network, $E_i(x)$ — i-й эксперт (обычно FFN).

8.2 Преимущества и недостатки

Преимущества:

- Больше параметров, те же FLOPs на токен.
- Качество близко к dense-модели с полным размером.
- Естественная специализация экспертов (разные эксперты для разных тем/задач).

Недостатки:

- Все эксперты в памяти — высокий VRAM.
- Load balancing: неравномерное распределение токенов по экспертам.
- Fine-tuning сложнее (overfitting, не все эксперты учатся).
- Коммуникационные overhead при распределённом обучении.

8.3 Практические реализации

- **Mixtral 8x7B (Mistral, 2023):** 8 экспертов, top-2 активация. 47B всего / 12.9B активных.
- **DeepSeekMoE:** мелкозернистые эксперты (больше маленьких), shared expert для всех токенов, auxiliary loss-free load balancing через динамический bias.
- **Switch Transformer (Google, 2021):** top-1 активация (один эксперт на токен). Максимальная спарсность.
- **GShard:** распределённый MoE для машинного перевода.

8.4 Технические детали DeepSeekMoE

- **Мелкозернистые эксперты:** 256 экспертов вместо 8-16, каждый меньше. Более гибкое распределение знаний.
- **Shared expert:** один общий FFN для всех токенов + routed эксперты для специализации.
- **Dynamic bias вместо aux loss:** регуляция загрузки экспертов через bias-терм в gating, без дополнительных потерь.

9 Выводы

- Языковое моделирование — предсказание следующего токена — основа всех современных LLM.
- Encoder-only (BERT) для понимания, decoder-only (GPT) для генерации.
- Scaling Laws: модель и данные должны масштабироваться согласованно.

- RLHF выравнивает модель под человеческие предпочтения.
- DeepSeek показала, что reasoning можно получать через RL.
- MoE — новый стандарт для масштабирования (больше параметров, те же FLOPs).